

Aula 05 — Spring Framework

Relacionamentos 1:N e N:N e Itens da venda

Mauricio Duailibi Neto

Turma Java Entra21

Como será a aula de hoje

Parte 1 — 18h30 às 20h

- Revisão da Aula 04
- Relacionamento: um cadastro aponta para outro
- 1:N com categoria e material
- N:N: a ideia da lista do meio
- Projeto **laboratorio** — fazemos juntos

Parte 2 — 20h20 às 22h

- Voltamos ao **StockSales**
- Venda com **vários** produtos

Dois projetos nesta aula

Não vamos misturar os códigos.

laboratorio Primeira parte da aula. Aqui aprendemos a ligar categoria e material.

StockSales O sistema da loja, que já existe. Só depois do intervalo.

Quando aparecer um código, o slide vai dizer:

qual projeto e **qual arquivo** estamos mexendo.

Mapa da aula

- 1 Revisão da Aula 04
- 2 Relacionamentos
- 3 Projeto laboratorio
- 4 Categoria (1:N)
- 5 Tela com select
- 6 Desafios
- 7 Parte 2 — StockSales
- 8 Recapitulando

O que já sabemos

- GET lista. GET com id devolve um só
- POST cria. PUT altera. DELETE apaga
- PUT **não** gera id novo e **não** dá add
- Retirar / vender é uma ação com regra
- Se a regra falhar, o servidor recusa e o JS usa catch

No StockSales, as listas ficam em Dados.

A venda de **um** produto já baixa o estoque.

Os quatro verbos

Ação	Verbo HTTP	O que faz
Listar / ler	GET	Lê, não muda nada
Cadastrar	POST	Cria um item novo
Editar	PUT	Troca os dados de um item
Excluir	DELETE	Apaga um item

Hoje esses verbos continuam iguais.

O que muda é o **conteúdo** do cadastro:
um objeto passa a guardar o id de outro.

O que a venda da aula passada já fazia

O JSON da venda era assim:

```
{"clienteId": 1, "produtoId": 2, "quantidade": 3}
```

clienteId e produtoId já são uma ligação.

A venda não copia o nome do cliente.

Ela guarda o **número** que aponta para ele.

Hoje vamos dar nome a isso e usar em mais lugares.

O que ainda falta

No laboratorio, o material tem id, nome e quantidade.
Ele não diz de que grupo é.

No StockSales, a venda só aceita **um** produto.
Na loja de verdade, a pessoa leva vários de uma vez.

Pergunta de hoje

Como um cadastro aponta para outro
sem copiar todos os dados?

Cadastros que se falam

Até agora cada lista vivia sozinha:
materiais de um lado, produtos de outro.

No mundo real eles se encaixam:

- O material **pertence** a uma categoria
- A venda **pertence** a um cliente
- A venda **tem** vários produtos

Essa ligação chama **relacionamento**.

Uma analogia que todo mundo já viu

Uma turma tem vários alunos.

Um aluno está em **uma** turma.

Não copiamos a lista de alunos dentro da turma
toda vez que alguém pergunta o nome da turma.

Guardamos no aluno o número da turma:

```
turmaId = 2.
```

Quando precisamos do nome, procuramos a turma 2
na lista de turmas.

Dois jeitos de ligar

Nome	Significado
1:N	Um de um lado, vários do outro
N:N	Vários dos dois lados

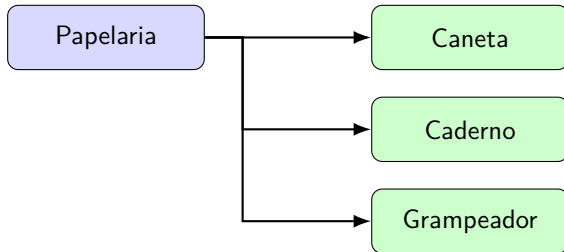
1:N — uma categoria, vários materiais.

Cada material tem **uma** categoria.

N:N — um material tem várias etiquetas,
e a mesma etiqueta serve para vários materiais.

1:N — o desenho

Uma categoria aponta para vários materiais.
O material aponta para **uma** categoria.



Papeleria é o “1”.
Caneta, Caderno e Grampeador são o “N”.

Como gravamos o 1:N

Não colocamos uma lista de materiais dentro da categoria.

Colocamos no material o id da categoria:

Material	quantidade	categoriaId
Caneta	10	1
Caderno	5	1
Mouse	3	2

A categoria 1 é Papelaria.

A categoria 2 é Informática.

É o mesmo jeito do `clienteId` na venda.

O JSON do material com categoria

O cadastro passa a levar mais um número:

```
{"nome": "Caneta", "quantidade": 10, "categoriaId": 1}
```

O servidor não precisa do texto “Papelaria” nesse JSON.

O id 1 já aponta para ela.

Se a pessoa mudar o nome da categoria depois,
a Caneta continua ligada ao id 1.

O nome novo aparece sozinho.

N:N — por que um id só não basta

Um material pode ser Frágil e Uso interno.
A etiqueta Uso interno serve para Caneta e Caderno.
Se o material tiver um único `etiquetaId`,
ele só cabe em uma etiqueta.
Se a etiqueta tiver um único `materialId`,
ela só cabe em um material.
Os dois lados são “vários”.
Um campo só não resolve.

N:N — a lista do meio

Criamos uma terceira lista.

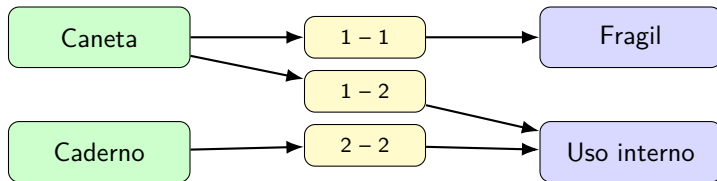
Cada linha é um par: este material + esta etiqueta.

materialId	etiquetaId
1 (Caneta)	1 (Fragil)
1 (Caneta)	2 (Uso interno)
2 (Caderno)	2 (Uso interno)

Caneta aparece duas vezes: tem duas etiquetas.

Uso interno aparece duas vezes: serve para dois materiais.

N:N — o desenho



O amarelo é a lista do meio.
Sem ela, os dois lados não se encontram.

Isso já aparece na loja

Uma venda tem vários produtos.

Um produto aparece em várias vendas.

Isso é N:N.

A lista do meio é o **item da venda**:

qual produto, quantas unidades, qual subtotal.

Na primeira parte treinamos a ideia no laboratório.

Depois do intervalo, a loja passa a aceitar
mais de um produto na mesma venda.

O que não vamos usar hoje

- Banco de dados
- Anotações do tipo `@OneToMany`
- JPA

Continuamos com lista na memória e um `int` guardando o id do outro cadastro.

O raciocínio é o mesmo que o banco vai usar depois. Só a ferramenta muda.

O projeto laboratorio

Continuamos no projeto **laboratorio**.
Não geramos outro no `start.spring.io`.

O que já deve estar pronto

Material com GET, POST, PUT e DELETE.
GET por id. Retirar quantidade. Formulário na tela.

Hoje entra um cadastro novo: **categoria**.
O material ganha um `categoriaId`.

Passo 1 — Subir o Spring

Onde estamos

Projeto **laboratorio**

No terminal, dentro da pasta laboratorio:

```
./mvnw spring-boot:run
```

No Windows: `.\mvnw.cmd spring-boot:run`

Abram no navegador: `http://localhost:8080`

Um Spring por vez

Se o StockSales ainda estiver na porta 8080, parem o outro antes.

Quais arquivos já existem

Projeto: **laboratorio**

Arquivo	Para quê
Material.java	Os dados de um material
MaterialController.java	GET, POST, PUT, DELETE
static/index.html	A página
static/js/app.js	O JavaScript da página

Vamos **acrescentar** categoria e a classe Dados.

Onde cada arquivo fica

```
laboratorio/  
  src/main/java/com/entra21/laboratorio/  
    LaboratorioApplication.java  
    Dados.java                (novo)  
    model/Material.java  
    model/Categoria.java      (novo)  
    controller/MaterialController.java  
    controller/CategoriaController.java (novo)  
  src/main/resources/static/  
    index.html  
    js/app.js
```

Por que a classe Dados

Vamos ter dois Controllers: material e categoria.

Se cada um guardar a própria lista, um **não vê** o cadastro do outro.

Vocês já fizeram isso no StockSales.

No laboratório é a mesma ideia: uma classe só, com as listas `static`.

`static` = uma lista só, visível para qualquer classe do projeto.

Classe Dados

Onde estamos

Projeto **laboratorio**

Arquivo novo:

src/main/java/com/entra21/laboratorio/Dados.java

```
package com.entra21.laboratorio;

import java.util.ArrayList;
import com.entra21.laboratorio.model.Material;
import com.entra21.laboratorio.model.Categoria;

public class Dados {
    public static ArrayList<Material> materiais = new ArrayList<>();
    public static int proximoIdMaterial = 1;
    public static ArrayList<Categoria> categorias = new ArrayList<>();
    public static int proximoIdCategoria = 1;
}
```

Mover a lista de materiais

Onde estamos

`MaterialController.java`

Tirem os campos `materiais` e `proximoId` de dentro da classe.

Em todos os métodos, troquem:

- `materiais` → `Dados.materiais`
- `proximoId` → `Dados.proximoIdMaterial`

O import: `com.entra21.laboratorio.Dados`

Confirmam GET, POST, PUT e DELETE no navegador **antes** de seguir. A lista precisa continuar funcionando.

Classe Categoria

Onde estamos

Arquivo novo:

src/main/java/com/entra21/laboratorio/model/Categoria.java

```
public class Categoria {  
    private int id;  
    private String nome;  
  
    public Categoria() {  
    }  
  
    public Categoria(int id, String nome) {  
        this.id = id;  
        this.nome = nome;  
    }  
}
```

Completem os gets e sets de id e nome.

categoriaId no Material

Onde estamos

model/Material.java

Acrescentem o campo e os get/set.

```
private int categoriaId;

public int getCategoriaId() {
    return categoriaId;
}

public void setCategoriaId(int categoriaId) {
    this.categoriaId = categoriaId;
}
```

O material agora aponta para uma categoria.

Construtor cheio do Material

Onde estamos

Ainda em `Material.java`

Se ainda não tiverem, acrescentem este construtor.

Mantenham o vazio. O vazio continua obrigatório no POST.

```
public Material(int id, String nome,  
                int quantidade, int categoriaId) {  
    this.id = id;  
    this.nome = nome;  
    this.quantidade = quantidade;  
    this.categoriaId = categoriaId;  
}
```

CategoriaController — seed e GET

Onde estamos

controller/CategoriaController.java, arquivo novo

```
@RestController
public class CategoriaController {

    public CategoriaController() {
        if (Dados.categorias.isEmpty()) {
            Dados.categorias.add(new Categoria(1, "Papeleria"));
            Dados.categorias.add(new Categoria(2, "Informatica"));
            Dados.categorias.add(new Categoria(3, "Limpeza"));
            Dados.proximoIdCategoria = 4;
        }
    }

    @GetMapping("/api/categorias")
    public ArrayList<Categoria> listar() {
        return Dados.categorias;
    }
}
```

POST de categoria

Onde estamos

Ainda em CategoriaController.java

```
@PostMapping("/api/categorias")
public Categoria cadastrar(@RequestBody Categoria categoria) {
    categoria.setId(Dados.proximoIdCategoria);
    Dados.proximoIdCategoria = Dados.proximoIdCategoria + 1;
    Dados.categorias.add(categoria);
    return categoria;
}
```

Testem no navegador:

<http://localhost:8080/api/categorias>

Devem aparecer Papelaria, Informatica e Limpeza.

PUT de material também grava a categoria

Onde estamos

MaterialController.java, método atualizar

Junto de setNome e setQuantidade:

```
Dados.materiais.get(i).setNome(dados.getNome());  
Dados.materiais.get(i).setQuantidade(dados.getQuantidade());  
Dados.materiais.get(i).setCategoriaId(dados.getCategoriaId());
```

Sem essa linha, o Editar apaga a categoria e deixa categoriaId em 0.

O POST de material já lê o campo novo

Não precisamos de método extra.

O JSON agora traz categoriaId.

O Spring preenche o campo no objeto.

O add na lista já guarda tudo.

Só o JavaScript precisa passar a **enviar** esse número.

Conferindo o backend

`/api/categorias` lista as três categorias

`/api/materiais` continua listando

Um POST de material com `categoriaId` no JSON devolve o campo preenchido

Se `/api/categorias` der 404, o Controller novo não compilou ou o Spring não reiniciou.

A pessoa não precisa decorar o id

Digitar `categoriaId = 2` na mão é fácil de errar.

Na aula passada, a venda já usava um `<select>`:
a pessoa escolhe o **nome**, o JS manda o **id**.

Vamos fazer o mesmo para a categoria do material.

O select no HTML

Onde estamos

Projeto **laboratorio**

static/index.html, dentro do form

```
<label>Categoria</label>  
<select id="categoriaId"></select>
```

Fica junto dos inputs de nome e quantidade.
O JS vai encher as opções depois do GET.

Uma lista de categorias no JS

Onde estamos

static/js/app.js, no topo do arquivo

Vamos guardar a resposta do GET.

A lista da tela usa essa variável para achar o nome.

```
var categorias = [];
```

Preencher o select

Onde estamos

static/js/app.js

```
function carregarCategorias() {  
  axios.get("/api/categorias").then(function (resposta) {  
    categorias = resposta.data;  
    var select = document.getElementById("categoriaId");  
    select.innerHTML = "";  
    for (var i = 0; i < categorias.length; i++) {  
      var opcao = document.createElement("option");  
      opcao.value = categorias[i].id;  
      opcao.textContent = categorias[i].nome;  
      select.appendChild(opcao);  
    }  
  });  
}
```

value é o id. O texto visível é o nome.

Por que value é o id

O que a pessoa vê: Papelaria.

O que o servidor precisa: 1.

O `<option>` guarda os dois:
`value="1"` e o texto Papelaria.

`select.value` devolve "1" — ainda é texto.

Por isso usamos `Number(...)` na hora de montar o JSON.

Achar o nome pelo id

Onde estamos

static/js/app.js

```
function nomeDaCategoria(categoriaId) {  
  for (var i = 0; i < categorias.length; i++) {  
    if (categorias[i].id === categoriaId) {  
      return categorias[i].nome;  
    }  
  }  
  return "";  
}
```

Percorre a lista. Quando o id bate, devolve o nome.
Se não achar, devolve texto vazio.

Mostrar o nome na lista

Onde estamos

static/js/app.js, no for de carregarMateriais

Troquem o `textContent` do `li` por:

```
li.textContent = material.nome  
  + " - quantidade: " + material.quantidade  
  + " - " + nomeDaCategoria(material.categoriaId)  
  + " ";
```

A pessoa lê “Caneta - quantidade: 10 - Papelaria”.

Não lê o número 1.

O submit manda o categoriaId

Onde estamos

static/js/app.js, no objeto dados
do submit — no POST e no PUT

```
var dados = {  
  nome: document.getElementById("nome").value,  
  quantidade: Number(  
    document.getElementById("quantidade").value),  
  categoriaId: Number(  
    document.getElementById("categoriaId").value)  
};
```

Sem Number, o Java pode recusar o campo,
porque espera int.

Editar também preenche o select

Onde estamos

static/js/app.js, no clique de Editar

Além de nome e quantidade:

```
document.getElementById("categoriaId").value =  
    evento.target.getAttribute("data-categoria-id");
```

No botão Editar, acrescentem:

```
btnEditar.setAttribute("data-categoria-id",  
    material.categoriaId);
```

Quando carregar a página

Onde estamos

No final de `app.js`

As categorias precisam chegar **antes** da lista, senão `nomeDaCategoria` não acha o nome.

```
carregarCategorias();  
carregarMateriais();
```

Se o nome da categoria sair vazio na primeira carga, chamem `carregarMateriais` **dentro** do `then` de `carregarCategorias`.

O caminho do cadastro com categoria

- 1 A página pede GET /api/categorias
- 2 O select mostra Papelaria, Informatica, Limpeza
- 3 A pessoa escolhe Papelaria, digita Caneta, envia
- 4 O JS manda categoriaId: 1 no POST
- 5 O material entra na lista com esse número
- 6 Na tela, o JS troca 1 por Papelaria

No Network: um GET de categorias e um POST de material.

Conferindo o 1:N

O select mostra as três categorias

Cadastrar Caneta em Papelaria mostra o nome na lista, não só o id

Editar troca a categoria e não duplica o material

Mouse em Informatica não aparece como Papelaria

No Network, o JSON do POST tem categoriaId

Erros que mais aparecem

- Select vazio — `carregarCategorias` não rodou
- Nome da categoria em branco — a lista de materiais rodou antes do GET de categorias
- `categoriaId` some no Editar — faltou o `setCategoriaId` no PUT
- Lista de materiais some — ainda está no Controller, não em Dados
- Porta 8080 ocupada

Agora é com vocês

Continuem no projeto **laboratorio**.
Mesmos arquivos.

- 1 Filtrar a lista por categoria
- 2 Não apagar categoria que ainda tem material
- 3 Etiquetas: o N:N

Salve → reinicie se precisar → teste no navegador.
F12, aba Network.

Desafio 1 — filtro por categoria

Onde estamos

`index.html` e `static/js/app.js`

Coloquem um segundo select, por exemplo
`id="filtro-categoria"`.

A primeira opção: `value=` e texto “Todas”.

As outras: as mesmas categorias.

Em `carregarMateriais`, se o filtro tiver um id,
só mostrem o material cujo `categoriaId` é aquele.

No `change` do select, chamem `carregarMateriais` de novo.

Desafio 1 — o if do filtro

Ideia do for (vocês escrevem o resto):

```
var filtro = document.getElementById(  
    "filtro-categoria").value;  
  
if (filtro !== "" &&  
    material.categoriaId !== Number(filtro)) {  
    continue;  
}
```

continue pula para o próximo material
e não cria o li.

Desafio 2 — não apagar categoria em uso

Onde estamos

`CategoriaController.java` e `app.js`

Antes de remover a categoria, percorram `Dados.materiais`.

Se algum material tiver aquele `categoriaId`, lancem `ResponseStatusException` com `HttpStatus.BAD_REQUEST`.

No JS, o clique de excluir categoria usa `catch` e escreve em `#mensagem`:
“Categoria em uso”.

Testem apagar Papelaria com a Caneta ainda cadastrada:
tem que recusar.

Desafio 3 — etiquetas (N:N)

Onde estamos

Arquivos novos no laboratorio

- 1 Classe Etiqueta: id e nome
- 2 Classe MaterialEtiqueta: materialId e etiquetaId
- 3 Listas dessas duas classes em Dados
- 4 GET e POST de /api/etiquetas
- 5 POST /api/materiais/{id}/etiquetas/{etiquetaId} para vincular

Seed sugerido: Fragil e Uso interno.

Desafio 3 — o POST de vincular

O método não cria material nem etiqueta.
Só acrescenta uma linha na lista do meio.

```
@PostMapping("/api/materiais/{id}/etiquetas/{etiquetaId}")
public MaterialEtiqueta vincular(@PathVariable int id,
    @PathVariable int etiquetaId) {
    MaterialEtiqueta ligacao = new MaterialEtiqueta();
    ligacao.setMaterialId(id);
    ligacao.setEtiquetaId(etiquetaId);
    Dados.ligacoes.add(ligacao);
    return ligacao;
}
```

Construtor vazio + gets e sets na classe MaterialEtiqueta.

Desafio 3 — mostrar na lista

No JS, peçam GET das ligações (um GET `/api/ligacoes` ajuda).

Para cada material, percorram as ligações.

Quando o `materialId` bater, achem o nome da etiqueta e cole no texto do `li`.

Um material pode mostrar duas etiquetas.

A mesma etiqueta pode aparecer em dois materiais.

Extra: botão para desvincular, apagando aquela linha da lista do meio — sem apagar o material.

Se terminarem os três, ainda no **laboratorio**:

- Não vincular a mesma etiqueta duas vezes no mesmo material
- Recusar vincular se o material ou a etiqueta não existir
- Página simples só para cadastrar categoria

Checklist

Estou no projeto laboratorio?
As listas estão em Dados?
/api/categorias devolve as três?
O material tem categoriaId?
A lista mostra o **nome** da categoria?
O PUT não zera a categoria?

Retomamos às **20h20**

Fechem o laboratório.

Parem o Spring.

Na segunda parte abrimos o **StockSales**.

Mudamos de projeto

Onde estamos

Agora é o projeto **StockSales**.
O laboratório já está encerrado.

Parem o Spring do laboratório.
Abram a pasta do StockSales e subam de novo.

Onde o StockSales parou

Na aula passada vocês chegaram em:

- Listas em Dados
- Produto com PUT
- Cliente com GET e POST
- Venda de **um** produto, com baixa de estoque

A venda ainda tem `produtoId` e `quantidade` nela mesma. Só cabe um produto.

O que vamos fazer no StockSales

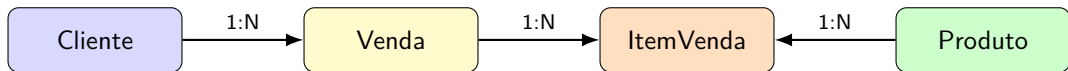
Uma venda com **vários** produtos.

- Quem compra: um cliente
- O quê: uma lista de itens
- Cada item: um produto + uma quantidade + um subtotal
- Total da venda = soma dos subtotais

É o N:N da primeira parte, com os nomes da loja.

A lista do meio se chama ItemVenda.

O desenho da venda



Um cliente tem várias vendas.

Uma venda tem vários itens.

Um produto aparece em vários itens.

Venda ↔ Produto é N:N.

ItemVenda é a ficha do meio.

O que sai da Venda e o que entra

Antes (aula 04)	Agora
produtoId na Venda	vai para o item
quantidade na Venda	vai para o item
um produto só	lista itens
total	continua na Venda

clienteId permanece na Venda:
a venda inteira é de um cliente só.

Classe ItemVenda

Onde estamos

Projeto **StockSales**

Arquivo novo:

src/main/java/com/entra21/stocksales/model/ItemVenda.java

```
public class ItemVenda {  
    private int produtoId;  
    private int quantidade;  
    private double subtotal;  
  
    public ItemVenda() {  
    }  
}
```

Gets e sets dos três atributos.

O subtotal o servidor calcula. O JS não manda.

Venda.java — a lista de itens

Onde estamos

model/Venda.java

Tirem `produtoId` e `quantidade`.

Acrescentem:

```
private ArrayList<ItemVenda> itens = new ArrayList<>();

public ArrayList<ItemVenda> getItens() {
    return itens;
}

public void setItens(ArrayList<ItemVenda> itens) {
    this.itens = itens;
}
```

Import: `java.util.ArrayList`.

Construtor vazio continua.

O JSON que o JavaScript vai mandar

Ana leva um Mouse e dois Teclados:

```
{  
  "clienteId": 1,  
  "itens": [  
    {"produtoId": 2, "quantidade": 1},  
    {"produtoId": 1, "quantidade": 2}  
  ]  
}
```

Sem total. Sem subtotal.

O servidor preenche depois, com o preço do produto.

O que o POST faz, em dois for

Se baixarmos o estoque do primeiro item e o segundo falhar, a loja já perdeu estoque à toa.

Por isso **dois** for:

- 1 Conferir cliente, produtos e estoque de **todos** os itens. Se um falhar, recusa e **não muda nada**
- 2 Só então: subtotal, total, baixa de estoque, gravar a venda

Achar um produto — método auxiliar

Onde estamos

VendaController.java

Vocês já repetiram esse for várias vezes.

Um método na mesma classe evita copiar de novo.

```
private Produto acharProduto(int id) {  
    for (int i = 0; i < Dados.produtos.size(); i++) {  
        if (Dados.produtos.get(i).getId() == id) {  
            return Dados.produtos.get(i);  
        }  
    }  
    return null;  
}
```

Façam o equivalente para cliente:
acharCliente.

POST — cliente e lista vazia

Onde estamos

VendaController.java, método cadastrar

```
@PostMapping("/api/vendas")
public Venda cadastrar(@RequestBody Venda pedido) {
    Cliente cliente = acharCliente(pedido.getClientId());
    if (cliente == null) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST, "Cliente nao encontrado");
    }
    if (pedido.getItems() == null || pedido.getItems().isEmpty()) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST, "Venda sem itens");
    }
}
```

O método ainda não acabou.

POST — primeiro for, só confere

Onde estamos

Ainda no mesmo método cadastrar

```
for (int i = 0; i < pedido.getItems().size(); i++) {  
    ItemVenda item = pedido.getItems().get(i);  
    Produto produto = acharProduto(item.getProdutoId());  
    if (produto == null) {  
        throw new ResponseStatusException(  
            HttpStatus.BAD_REQUEST, "Produto nao encontrado");  
    }  
    if (produto.getEstoque() < item.getQuantidade()) {  
        throw new ResponseStatusException(  
            HttpStatus.BAD_REQUEST, "Estoque insuficiente");  
    }  
}
```

Nenhum setEstoque aqui.

Se lançar a exceção, o segundo for nem roda.

POST — segundo for, grava de verdade

Onde estamos

Ainda no mesmo método cadastrar

```
double total = 0;
for (int i = 0; i < pedido.getItems().size(); i++) {
    ItemVenda item = pedido.getItems().get(i);
    Produto produto = acharProduto(item.getProdutoId());
    item.setSubtotal(produto.getPreco() * item.getQuantidade());
    total = total + item.getSubtotal();
    produto.setEstoque(
        produto.getEstoque() - item.getQuantidade());
}
pedido.setId(Dados.proximoIdVenda);
Dados.proximoIdVenda = Dados.proximoIdVenda + 1;
pedido.setTotal(total);
Dados.vendas.add(pedido);
return pedido;
}
```

Olhando o segundo for com calma

- 1 Pega o item da posição `i`
- 2 Acha o produto daquele `produtoId`
- 3 $\text{Subtotal} = \text{preço} \times \text{quantidade}$ daquele item
- 4 Soma no total da venda
- 5 Diminui o estoque daquele produto

O `for` anda item por item.

Mouse e Teclado baixam estoque os dois.

O `GET /api/vendas` continua igual:
devolve `Dados.vendas`. O JSON agora traz `itens`.

A tela precisa de um carrinho

Na aula passada: um select de produto e Confirmar.

Agora a pessoa **acrescenta** produtos um a um e só depois confirma.

No JavaScript isso é uma lista:

```
var itens = [];
```

Cada clique em Adicionar faz push de um objeto com `produtoId` e `quantidade`.

Confirmar manda essa lista no POST.

Onde estamos

static/vendas-nova.html

Mantenham o select de cliente.

O de produto e a quantidade servem para **Adicionar**,
não para confirmar direto.

```
<label>Produto</label>
<select id="produtoId"></select>
<label>Quantidade</label>
<input id="quantidade" type="number">
<button id="btn-adicionar" type="button">Adicionar</button>
<ul id="lista-itens"></ul>
<button id="btn-vender" type="button">Confirmar venda</button>
<p id="mensagem"></p>
```

A lista itens no JS

Onde estamos

static/js/vendas-nova.js, no topo

```
var itens = [];
```

carregarClientes e carregarProdutos
continuam iguais à aula passada.

Mostrar o carrinho na página

Onde estamos

vendas-nova.js

```
function mostrarItens() {  
  var lista = document.getElementById("lista-itens");  
  lista.innerHTML = "";  
  for (var i = 0; i < itens.length; i++) {  
    var li = document.createElement("li");  
    li.textContent = "Produto " + itens[i].produtoId  
      + " - qtd " + itens[i].quantidade;  
    lista.appendChild(li);  
  }  
}
```

Extra depois: trocar o id pelo nome, percorrendo os produtos que o GET já trouxe.

Onde estamos

vendas-nova.js

```
document.getElementById("btn-adicionar")
  .addEventListener("click", function () {
    var item = {
      produtoId: Number(
        document.getElementById("produtoId").value),
      quantidade: Number(
        document.getElementById("quantidade").value)
    };
    itens.push(item);
    document.getElementById("quantidade").value = "";
    mostrarItens();
  });
```

push põe no fim da lista.

A venda ainda **não** foi para o servidor.

Confirmar a venda

Onde estamos

vendas-nova.js, no clique de Confirmar

```
var dados = {
  clienteId: Number(document.getElementById("clienteId").value),
  itens: itens
};

axios.post("/api/vendas", dados)
  .then(function () {
    document.getElementById("mensagem").textContent =
      "Venda registrada";
    itens = [];
    mostrarItens();
    carregarProdutos();
  })
  .catch(function () {
    document.getElementById("mensagem").textContent =
      "Nao foi possivel registrar a venda";
  });
```

Por que esvaziar itens depois do then

A venda já foi gravada.

Se não fizermos `itens = []`, o próximo Confirmar manda os mesmos produtos de novo.

`carregarProdutos` atualiza o estoque no select.
Sem isso, a tela ainda mostra a quantidade antiga.

Histórico — percorrer os itens

Onde estamos

static/js/vendas.js

A venda agora tem uma lista dentro.

Um for na venda e outro nos itens:

```
for (var i = 0; i < vendas.length; i++) {  
    var texto = "Cliente " + vendas[i].clienteId  
        + " - total R$ " + vendas[i].total + " - ";  
    var itensVenda = vendas[i].itens;  
    for (var j = 0; j < itensVenda.length; j++) {  
        texto = texto + "prod " + itensVenda[j].produtoId  
            + " x" + itensVenda[j].quantidade + " ";  
    }  
    var li = document.createElement("li");  
    li.textContent = texto;  
    lista.appendChild(li);  
}
```

O caminho da venda com vários itens

- 1 A pessoa escolhe Ana, Mouse, quantidade 1, Adicionar
- 2 Escolhe Teclado, quantidade 2, Adicionar
- 3 A <u1> mostra os dois
- 4 Confirmar manda o JSON com `items`
- 5 O servidor confere os dois, depois baixa os dois estoques
- 6 O histórico lista a venda com total e os itens

Dá para adicionar dois produtos antes de confirmar

A venda grava os dois itens

O estoque dos dois produtos diminui

Pedir mais do que o estoque de **um** deles recusa a venda **inteira**

O outro produto **não** perde estoque nessa recusa

Confirmar com carrinho vazio recusa

Como conferir a recusa inteira

Mouse com estoque 30. Headset com estoque 15.

Adicionem Mouse quantidade 1 e Headset quantidade 20.
Confirmar.

A mensagem de erro aparece.

Recarreguem produtos: Mouse continua 30.

Headset continua 15.

Se o Mouse tiver baixado, o primeiro for de conferência
não está antes do `setEstoque`.

Se der tempo:

- No carrinho e no histórico, mostrar o **nome** do produto, não só o id
- Botão Remover em cada item do carrinho, antes de confirmar
- Não permitir quantidade 0 ou negativa no Adicionar
- Recusar dois itens com o mesmo `produtoId` na mesma venda — ou somar as quantidades

O que não entra hoje

- Cancelar venda e devolver estoque
- Banco de dados
- Login
- Anotações JPA

Meta no StockSales

Uma venda com vários itens.

Estoque de cada produto baixando.

Se um item falhar, a venda inteira recusa.

Critérios de aceite

Venda aceita dois produtos diferentes

total é a soma dos subtotais

Estoque de cada produto diminui a quantidade daquele item

Estoque insuficiente em um item recusa tudo

Histórico lista os itens

Home ainda leva a produtos, clientes, nova venda e histórico

O que vimos hoje

- 1 Relacionamento é um cadastro apontando para outro
- 2 1:N: o lado “muitos” guarda o id do lado “um”
- 3 N:N: uma terceira lista guarda os dois ids
- 4 O select mostra o nome e manda o id
- 5 No StockSales, ItemVenda é essa lista do meio

Os dois desenhos, lado a lado

laboratorio	StockSales
Categoria 1:N Material categoriaId no material	Cliente 1:N Venda clienteId na venda
Etiqueta N:N Material MaterialEtiqueta	Produto N:N Venda ItemVenda

Mesma ideia. Nomes diferentes.

Aula 06

- Validação dos dados que chegam
- Erros HTTP com mensagem mais clara
- Organizar o que já cresceu no StockSales

Tragam o StockSales com venda de vários itens funcionando.

Obrigado!

Mauricio Duailibi Neto
Turma Java Entra21